

Introduction to Terraform

Software Engineering 2025-2026

The Cloud

■ The Cloud Is Everywhere

■ The Numbers

- ~9/10 organizations are using the cloud¹
 - ~7/10 use more than one¹
 - Half pursuing cloud-first strategy¹
 - Cloud market worth >\$900B+²
-



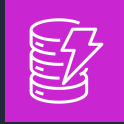
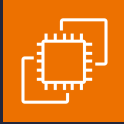
■ Why Cloud?

- Pay as you go
- Scale up or down in minutes
- Deploy for the whole globe
- Reduced operational burden



Building In The Cloud

■ AWS



...

...

...

What about a
hybrid cloud?

■ Azure



...

...

...

Furthermore

- Each provider has its own
 - API
 - CLI
 - Console
- Reproducibility?
- Version control??
- Disaster recovery???
- Oversight???
- ...
- ...
- ...

↓ You 🤖 (probably) ↓

- *"WHOA!"*
- *"I dont like this!"*
- *"This is getting too complex!"*
- *"I can't manually manage all of this!"*
- *"I should have become a farmer instead?!"*

BUT FEAR NOT!

 Introducing



Terraform

■ What is Terraform?

- Infrastructure as Code (IaC) tool
 - Manages **resources**
-

■ What's a resource?

Anything can be a resource

- Commonly: cloud resources
 - Databases
 - Object Storage
 - Serverless functions
 - Container services
 - Compute instances
 - But also
 - Proxmox VMs
 - TLS certificates
 - Spotify playlists³
 - Discord servers⁴
 - Domino's pizza orders⁵
-

■ Why Infrastructure as Code?

- Reproducible
- Version controlled
- Reviewable
- Self-documenting
- Automated
- Recoverable

Terraform Providers

■ What are Terraform Providers?

- Terraform "*plugins*"
- Manage **resources** of a **service**
- Provides data sources

■ Example providers

 AWS



hashicorp/aws

 Azure



hashicorp/azurerm

 GCP



hashicorp/google

 Domino's



ndmckinley/dominos⁵

■ Example resources

- `aws_s3_bucket`
- `aws_lambda_function`
- `aws_dynamodb_table`

■ Example data sources

- `aws_region`
- `aws_caller_identity`
- `aws_ip_ranges`

Terraform Basics

■ Configuration files

- Configuration directory
 - Automatic import
- `.tf` extension
- Can contain
 - Providers
 - Resources
 - Data sources
 - Variables
 - Locals
 - Outputs
- Lock file
- State file

■ Initialize Terraform

```
terraform init
```

■ Deploy resources

```
terraform apply
```

■ Configuring a provider

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.92"  
    }  
  }  
  
  provider "aws" {  
    region = "eu-central-1"  
  }  
}
```

■ Creating a resource

```
resource "aws_dynamodb_table" "my_table" {  
  name       = "my_table"  
  billing_mode = "PAY_PER_REQUEST"  
  hash_key   = "my_column"  
  
  attribute {  
    name = "my_column"  
    type = "S"  
  }  
}
```

Real world example

■ jAilbreak Software Engineering project

■ Our terraform config

```

└─ jailbreak
   └─ terraform
      ├── api_gateway.tf
      ├── api_lambda.tf
      ├── cloudfront.tf
      ├── cloudwatch_logs.tf
      ├── cognito.tf
      ├── dynamodb.tf
      ├── main.tf
      ├── outputs.tf
      ├── s3.tf
      └── variables.tf

```

■ We know our cloud resources!

Type	Count
IAM Role Policy	5
Lambda Permission	4
Lambda Function	4
DynamoDB Table	3
S3 Bucket	2
IAM Role	2
Cognito User Pool Client	2
CloudWatch Log Group	2
CloudFront Function	2
S3 Bucket Website Configuration	1
S3 Bucket Policy	1
Random String	1
Cognito User Pool Domain	1
Cognito User Pool	1
Cognito User Group	1
CloudFront Origin Access Control	1
CloudFront Distribution	1
API Gateway Stage	1
API Gateway REST API	1
API Gateway Deployment	1

■ Usage

- AWS provider
- Deployment of
 - Frontend
 - Backend
- Remote state
- Outputs

=> 37 Terraform managed resources

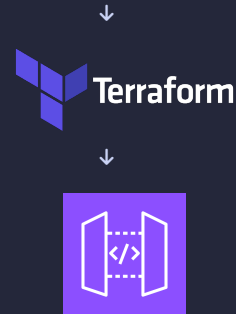
Real world example

■ Benefits for jAilbreak

- Had to switch AWS account
 - Switched in ~1 hour
- Ease of deployment
- Rollback temporary changes
- Tracked in Git
 - Merging of frontend/backend
- Automatic API routing

■ Automated API route creation

```
api_routes! {  
  ["levels"] {  
    GET --> levels::get_levels;  
  }  
  ["levels", (level_id), "validate"] {  
    POST --> levels::validate_password;  
  }  
  ...  
}
```



```
aws sso login  
terraform init  
terraform plan -lock=false
```

snippet +exec is disabled, run with -x to enable

Hands-On Live Demo

Demo Goals

- Install Terraform
- Initialize GitHub provider
- Create GitHub Repo
- Obtain the default branch
- Commit a README
- Obtain the commit hash
- Publish a release
- Disaster recovery
- Tear down deployment

Setting up

■ Required tools

■ Terraform

```
# Windows
winget install HashiCorp.Terraform

# MacOS
brew tap hashicorp/tap
brew install hashicorp/tap/terraform

# Linux
sudo pacman -S terraform # Arch
nix shell nixpkgs#terraform # Nix
```

■ Github

- Personal Access Token
 - <https://github.com/settings/tokens/new>
- Permissions
 - `repo`
 - Full access
 - `delete_repo`

Provider Setup

■ Configuring the provider

 demo.tf

```
1 terraform {
2   required_providers {
3     github = {
4       source = "integrations/github"
5       version = "6.12.1"
6     }
7   }
8 }
9
10 variable "github_token" {}
11
12 provider "github" {
13   token = var.github_token
14 }
```

 auth.auto.tfvars

```
1 github_token = "<your github access token>"
```

■ Initializing Terraform

```
terraform init
```

snippet +exec is disabled, run with -x to enable

Creating a repository

■ Define repository

 demo.tf

```
1 resource "github_repository" "live_demo" {  
2   name      = "terraform-live-demo"  
3   description = "Terraform-managed Repo"  
4   visibility = "public"  
5   auto_init  = true  
6 }
```

■ Apply changes

```
terraform apply
```

snippet +exec is disabled, run with -x to enable

Adding a README

■ Obtain default branch and define file

demo.tf

```
1 data "github_repository" "live_demo" {
2   name = github_repository.live_demo.name
3 }
4
5 resource "github_repository_file" "readme" {
6   repository      = github_repository.live_demo.name
7   branch          = data.github_repository.live_demo.default_branch
8   file            = "README.md"
9   content         = "# Hello from Terraform"
10  overwrite_on_create = true
11 }
```

■ Apply changes

```
terraform apply
```

snippet +exec is disabled, run with -x to enable

Obtaining the commit hash

■ Defining locals and outputs

 demo.tf

```
1 locals {  
2   readme_commit = github_repository_file.readme.commit_sha  
3 }  
4  
5 output "readme_commit" {  
6   value = local.readme_commit  
7 }
```

■ Apply changes and view outputs

```
terraform apply
```

snippet +exec is disabled, run with -x to enable

■ View current outputs

```
terraform output
```

Publishing a release

■ Declaring the release

demo.tf

```
1 resource "github_release" "one_dot_o" {
2   repository      = github_repository.live_demo.name
3   tag_name        = "v1.0.0"
4   target_commitish = local.readme_commit
5   draft           = false
6   prerelease      = false
7 }
```

■ Apply changes

```
terraform apply
```

snippet +exec is disabled, run with -x to enable

Disaster recovery

■ Example scenario

Greg the intern deleted super important 1.0 release

- Our multi million dollar SaaS is down
- Customers furious
- 30 missed calls from boss

■ Repair infrastructure

```
terraform apply
```

snippet +exec is disabled, run with -x to enable

Tear Down Infrastructure

■ Deleting all resources

```
terraform apply -destroy
```

snippet +exec is disabled, run with -x to enable

Thank You For Participating!

Footnotes

- ¹<https://www.oreilly.com/radar/the-cloud-in-2021-adoption-continues/>
- ²<https://www.grandviewresearch.com/industry-analysis/cloud-computing-industry>
- ³<https://registry.terraform.io/providers/conradludgate/spotify/latest>
- ⁴<https://registry.terraform.io/providers/Lucky3028/discord/latest>
- ⁵<https://github.com/nat-henderson/terraform-provider-dominos>